Forth — Reference

Forth here is a *compiler*, not the usual interpreter: each `: word … ;` definition compiles to a real .NET method (`lex` + `yacc` → C → `cc` → IL), with **no inner interpreter / threaded code**. The data stack is a real `.NET Stack<object>`, so `int`, `double`, and `string` can all sit on it and the arithmetic/print words are polymorphic.

```
forth prog.fth -o prog.exe      # compile to a native .NET exe
prog.exe                        # run it
```

Quick Reference

```
42  3.14  S" text"     \ push an int / double / string
+ - * / MOD            \ arithmetic (polymorphic: int, double, or string-concat for +)
DUP DROP SWAP OVER ROT ?DUP NIP TUCK 2DUP 2DROP DEPTH      \ stack juggling
= <> < > ≤ ≥ 0= 0< 0> AND OR XOR INVERT                    \ comparison / logic (true = -1)
. .S EMIT CR SPACE SPACES TYPE      ." text"               \ output
: name ... ;           \ define a word (a method)
IF ... ELSE ... THEN     BEGIN ... UNTIL     BEGIN ... WHILE ... REPEAT
n m DO ... LOOP   ( I = index, J = outer index )    ... +LOOP
VARIABLE v   v @   x v !        42 CONSTANT answer
\ comment to end of line ;  ( ... ) inline comment
```

Data types

Cells hold any of three runtime types (the stack is `Stack<object>`):

Type	.NET	Pushed by
int	Int32	`42`
double	Double	`3.14`
string	String	`S" hello"`

Operators are polymorphic by runtime type: `+` adds two numbers (promoting to double if either is) or **concatenates** if a string is involved; `.` prints whatever is on top. Comparisons push Forth booleans (`-1` true, `0` false).

Statements / Commands

Forth is postfix: words consume operands from the stack and push results. Control words: `IF/ELSE/THEN`, `BEGIN/UNTIL`, `BEGIN/WHILE/REPEAT`, `DO/LOOP` and `DO/+LOOP` (with loop indices `I`, `J`). Definitions: `: name … ;`. Storage: `VARIABLE`, `CONSTANT`, `@` (fetch), `!` (store).

Functions

A "function" is a colon definition `: name … ;`, compiled to a .NET method; it may recurse and may call words defined later. Words have no declared arity or types — they just act on the stack. Primitive words (`+`, `DUP`, `.`, …) are built in.

Input / Output

`.` prints the top cell (with a trailing space); `.S` prints the whole stack non-destructively; `EMIT` prints a character code; `CR` a newline; `SPACE` / `SPACES` spaces; `." text"` prints a literal; `TYPE` prints a string from the stack.

Graphics

None. Activity 8 below is **text art** built from `." "` / `CR`.

Notes

- Each `: word ;` is a real `public static` .NET method — so a compiled Forth file is referenceable from C#/VB.NET (words take/return no fixed types, operating on the shared stack).
- The stack is a `.NET Stack<object>`; value types are boxed, so mixed-type stacks and polymorphic operators "just work."
- A program is a file of definitions plus top-level words (run in order). There is no interactive REPL in the compiled model.

Subset boundaries

A solid core, not full ANS Forth. Not included: an array/cell-array word set (`CREATE` / `ALLOT` / `CELLS`), the return stack words (`>R` / `R>`), `DOES>`/defining words, `IMMEDIATE`/compile-time metaprogramming, `ABORT`/exceptions, and floating-point formatting words. `VARIABLE` gives single cells, not arrays.

Tutorial

Every example was compiled with `forth` and run; the output shown is real.

1. Your first program

```
." Hello, Forth!" CR
```

```
Hello, Forth!
```

A program is words evaluated left to right. `." …"` prints a literal string; `CR` prints a newline.

2. Variables and data types

```
42 CONSTANT answer
." answer=" answer . CR
VARIABLE x
7 x ! x @ . CR
." stack " 1 2 3 .S CR
```

```
answer=42
7
stack <3> 1 2 3
```

`CONSTANT` names a value; `VARIABLE` reserves a cell you write with `!` and read with `@`. `.S` shows the whole stack (`<3>` = three items). Numbers, doubles and strings all share the one stack.

3. Flow control

```
." until: " 0 BEGIN DUP . 1+ DUP 5 > UNTIL DROP CR
." while: " 1 BEGIN DUP 4 ≤ WHILE DUP . 1+ REPEAT DROP CR
." loop:  " 10 0 DO I . LOOP CR
```

```
until: 0 1 2 3 4 5
while: 1 2 3 4
loop:  0 1 2 3 4 5 6 7 8 9
```

`BEGIN/UNTIL` loops until the flag is true; `BEGIN/WHILE/REPEAT` loops while the flag holds; `DO/LOOP` counts with index `I`. (`IF/ELSE/THEN` works the same way on a flag.)

4. Arrays

Forth has no array type in this subset (see *Subset boundaries*) — the idiom is to loop over an index range with `DO/LOOP`, or to reserve single cells with `VARIABLE`. For example, summing the squares 1..5 without storage:

```
: sumsq 0 6 1 DO I I * + LOOP ;
sumsq . CR
```

```
55
```

`DO/LOOP` walks `I` from 1 to 5; each pass pushes `I*I` and adds it to the running total already on the stack.

5. Subroutines and functions

```
: square DUP * ;
6 square . CR
: fib DUP 2 < IF DROP 1 ELSE DUP 1 - fib SWAP 2 - fib + THEN ;
." fib(10)=" 10 fib . CR
```

```
36
fib(10)=89
```

A colon definition is a function compiled to a .NET method. `square` duplicates the top and multiplies; `fib` recurses (a word may call itself).

6. Memory management

Forth's memory model *is* the stack:

- **The data stack** (a `.NET Stack<object>`) holds operands; every word pops its inputs and pushes its outputs. `2 3 +` leaves `5`.
- `VARIABLE` reserves a single cell in a side table; `x !` stores, `x @` fetches.
- Value cells are boxed by .NET; strings are managed references. There is nothing to free — the garbage collector reclaims discarded values.

`.S` is the tool for seeing the stack while you reason about it.

7. Strings and a text layout

```
S" string via S-quote" TYPE CR
: greet ." Hi, " TYPE ." !" CR ;
S" Ada" greet
```

```
string via S-quote
Hi, Ada!
```

`S" …"` pushes a string; `TYPE` prints one from the stack — so a word like `greet` can take a string operand and lay out text around it.

8. Drawing a picture

No graphics — here is **text art**, a triangle from nested loops:

```
: stars 0 DO ." *" LOOP CR ;
: tri 5 1 DO I stars LOOP ;
tri
```

```
*
**
***
****
```

`stars` prints `n` asterisks then a newline; `tri` calls it for `I` = 1..4.

9. Where to go next

Each word is a real .NET method, so a compiled `.fth` file is a library C#/VB.NET can call. From here: build deeper stack vocabularies, lean on the polymorphic stack to mix numbers and strings, and read the generated C to see how `: … ;` becomes a method and how the control words lower to branches. Array/return-stack words are the natural next additions (see *Subset boundaries*).